



Retail Sales Analysis

1,000 transactions, ₹456,000 in revenue — a full MySQL breakdown

Explore

Clean

Query

Aggregate

Subquery

Swipe to see the solved problems →

Project Overview

Retail transaction data analyzed end-to-end in MySQL — exploration, cleaning, and business-question queries.



9 Columns

Transaction ID, Date, Customer, Gender, Age, Category, Qty, Price, Total



1,000 Records

Verified total transaction count, zero nulls



3 Categories

Beauty, Clothing, Electronics



MySQL

Workbench / CLI, standard SQL only

Workflow: Database Exploration → Data Cleaning → Business Queries → Aggregation → Subqueries

Solved Problem — Data Quality Checks

Before analysis, every dataset needs a trust check.



NULL check

Scanned all 9 key columns for missing values before trusting any aggregate.



Duplicate Transaction IDs

GROUP BY + HAVING COUNT(*) > 1 to catch repeated transaction records.



Invalid values

Flagged Age, Quantity, Price, and Total Amount that were zero or negative.

The Headline Numbers

Simple SUM and AVG aggregates — the first thing any stakeholder asks for.



₹456,000

Total revenue across 1,000 transactions



₹456

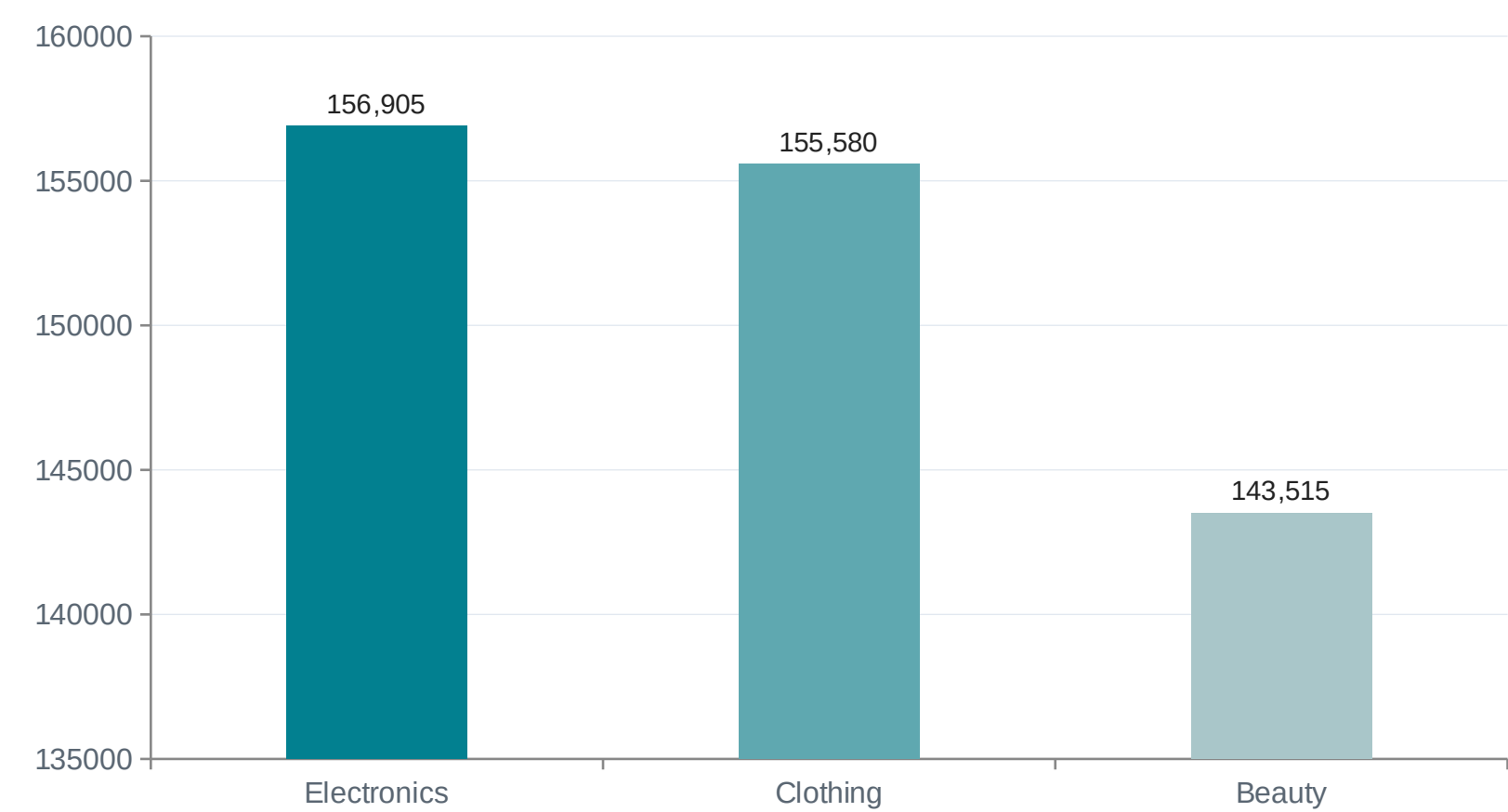
Average order value

```
SELECT SUM(`Total Amount`) AS Total_revenue  
FROM retail_sales_dataset;
```

Two one-line queries, two numbers every retail dashboard opens with.

Which Category Wins?

Revenue grouped by Product Category, ranked highest to lowest.



Electronics narrowly leads at ₹156,905 — just ₹1,325 ahead of Clothing

Who's Buying?

Transaction count grouped by gender — a near-even split.



510

Female transactions (51%)



490

Male transactions (49%)

```
SELECT Gender, COUNT(*) AS Total_Transactions
FROM retail_sales_dataset GROUP BY Gender;
```

A 1,000-row dataset split almost exactly down the middle by gender.

Solved Problem — Top 5 Transactions

ORDER BY + LIMIT to surface the highest-value single sales.

```
SELECT * FROM retail_sales_dataset ORDER BY `Total Amount` DESC LIMIT 5;
```

104

-- Total Revenue --

105

•

SELECT SUM(`Total Amount`) AS Total_revenue

106

FROM retail_sales_dataset;

100%

26:94

Result Grid

Filter Rows:

Q

Search

Export:

Total_...
456000



All top 5 transactions land at exactly ₹2,000 — a spend ceiling, likely the max qty × max price combo in the data

Solved Problem — Highest-Spending Customers

Nested subquery: group spend by customer, then match against the max.

```
SELECT `Customer ID`, SUM(`Total Amount`) AS Total_Spent
FROM retail_sales_dataset GROUP BY `Customer ID`
HAVING SUM(`Total Amount`) =
    (SELECT MAX(total_spent) FROM (...) AS customer_sales);
```

```
160 -- Customer Who Spent the Highest Total Amount --
161 • SELECT `Customer ID`,
162       SUM(`Total Amount`) AS Total_Spent
163 FROM retail_sales_dataset
164 GROUP BY `Customer ID`
165 HAVING SUM(`Total Amount`) =
166 (
167     SELECT MAX(total_spent)
168     FROM
169     (
170         SELECT `Customer ID`,
171               SUM(`Total Amount`) AS total_spent
172         FROM retail_sales_dataset
173         GROUP BY `Customer ID`
174     ) AS customer_sales
175 );
```

100% 31:156

Result Grid Filter Rows: Search Export:

Customer ID	Total_Spent
CUST015	2000
CUST065	2000
CUST072	2000
CUST074	2000
CUST089	2000
CUST093	2000
CUST109	2000
CUST118	2000

Result 39



Multiple customers tie for the top spend — ₹2,000, matching the same ceiling seen in the top 5 transactions



Let's Talk Data.

Full SQL script, README, and query breakdowns are live on GitHub. Always open to feedback and data conversations.



GitHub

Retail-Sales-SQL-Project repository



LinkedIn

Connect for more data analytics projects